

Доклад к защите ВКР — версия с отметками для сокращения

*«Разработка веб-приложения для организации личного файлового хранилища»
(LocalCloud)*

Полная версия ≈ 9 мин · без отмеченного ≈ 6–6,5 мин

Условные обозначения: обычный чёрный текст — основа, читается всегда; ✂
серый курсив — можно опустить, если не успеваете по времени.

Слайд 1 — Титульный

Уважаемый председатель и члены государственной экзаменационной комиссии! Вашему вниманию представляется выпускная квалификационная работа на тему «Разработка веб-приложения для организации личного файлового хранилища». Работа выполнена студентом группы ПИЖ-б-о-22-1 Магомедовым Имраном. ✂
Руководитель — кандидат технических наук, доцент Самойлов Филипп Владимирович.

Слайд 2 — Актуальность

Объём личных данных пользователей постоянно растёт, а основным способом их хранения остаются публичные облака — Google Drive, Dropbox, Яндекс Диск. Однако их подписочная модель означает постоянные и растущие платежи, а сами данные оказываются под контролем стороннего провайдера. ✂ *Провайдер может изменить тарифы, заблокировать аккаунт или прекратить обслуживание, а после 2022 года добавились и трудности с оплатой.* Альтернатива — самостоятельное размещение, но существующие решения, такие как Nextcloud и Seafile, пропускают весь файловый трафик через серверное приложение и потому требуют мощных серверов. При этом доступный частному пользователю сервер — это, как правило, бюджетный VPS с одним ядром и одним гигабайтом памяти. ✂ *Отсюда инженерная задача: организовать хранилище, передающее объёмы данных,*

многократно превышающие память сервера. Этим и определяется актуальность работы.

Слайд 3 — Цель и задачи

Цель работы — разработать веб-приложение LocalCloud, которое обеспечивает загрузку и скачивание больших файлов, иерархию данных, разграничение доступа между пользователями и публикацию файлов по ссылкам и при этом сохраняет работоспособность на сервере минимальной конфигурации. Для достижения цели решались пять задач. ∞ *Это анализ предметной области и формирование требований; проектирование архитектуры, базы данных и API; реализация серверной и клиентской частей; тестирование; и технико-экономическое обоснование с рассмотрением охраны труда.*

Слайд 4 — Заинтересованные лица (акторы)

В системе выделены четыре актора. Гость может войти, подать заявку на регистрацию или открыть публичную ссылку. Пользователь управляет своими файлами, папками и корзиной, выдаёт доступ другим пользователям и создаёт публичные ссылки. ∞ *Администратор дополнительно управляет учётными записями, квотами и журналом аудита.* Отдельный системный актор — фоновый обработчик, который выполняет отложенные операции из очереди задач.

Слайд 5 — Основной функционал

Основной функционал включает работу с файлами и папками произвольной вложенности, многочастичную загрузку напрямую в объектное хранилище и скачивание архивами. Реализованы корзина с восстановлением, общий доступ между пользователями с пятью уровнями прав и наследованием, а также публичные ссылки со сроком

действия и паролем. *⌘ Кроме того, поддерживаются квоты, генерация превью изображений, PDF и видео и панель администратора.*

Слайд 6 — Нефункциональные требования

Ключевое нефункциональное требование — производительность: работа на одном ядре и одном гигабайте памяти, причём размер файла не должен ограничиваться памятью сервера. Безопасность строится на токенах JWT в httpOnly-cookie с ротацией и хранении паролей только в виде хэшей. *⌘ Надёжность обеспечивается отсутствием потери данных при перезапуске и повтором фоновых задач. ⌘ Развёртывание выполняется одной командой.*

Слайд 7 — Архитектура системы

⌘ Архитектура спроектирована по модели C4. Система состоит из одностраничного приложения в браузере и шести контейнеров на сервере: шлюза nginx, API на FastAPI, фонового worker'а, базы данных PostgreSQL и хранилища MinIO. Ключевое архитектурное решение показано пунктиром: файловый трафик идёт напрямую от браузера к MinIO по подписанным ссылкам, в обход API. Именно поэтому потребление ресурсов сервера не зависит от размера передаваемых файлов.

Слайд 8 — Слои серверного приложения

Серверное приложение построено по слоям. Маршрутизаторы отвечают за валидацию и проверку прав, доменные сервисы содержат бизнес-логику, а репозитории инкапсулируют SQL. *⌘ Транзакциями управляет паттерн Unit of Work с автоматическим откатом при*

выходе без фиксации, а доступ к хранилищу скрыт за единым фасадом. Зависимости направлены строго к нижним слоям, циклов нет.

Слайд 9 — Паттерны проектирования

В проекте системно применён ряд паттернов проектирования: Repository и Unit of Work для слоя данных, Facade для хранилища и Adapter для асинхронной обёртки MinIO. *✂ Также использованы Dependency Injection через механизм FastAPI, связка Command и Registry для обработчиков фоновых задач и Proxu для ленивой загрузки настроек.*

Слайд 10 — Проектирование базы данных

База данных насчитывает четырнадцать таблиц и нормализована до третьей нормальной формы. *✂ Данные разделены на три контура — файловой иерархии, доступа и служебный. Файлы и папки обобщены таблицей узла со спутниками, а содержимое файлов хранится не в базе, а в объектном хранилище и адресуется ключом объекта.*

Слайд 11 — Технологический стек

Серверная часть написана на Python 3.13 и FastAPI с асинхронным стеком SQLAlchemy 2 и asyncpg. Клиентская часть — на TypeScript, React 19 и Vite. *✂ Метаданные хранит PostgreSQL 16, содержимое файлов — MinIO, а весь стек из шести сервисов разворачивается средствами Docker Compose.*

Слайд 12 — Структура проекта

Проект организован как монорепозитарий: серверная и клиентская части вместе с инфраструктурой версионированы совместно. *✂ Серверный код разделён по слоям в точном соответствии со*

спроектированной архитектурой, что подтверждает диаграмма пакетов — обратных зависимостей и циклов в ней нет.

Слайд 13 — Прошедшие проектные этапы

Разработка велась по упрощённой модели Git Flow с постоянными ветками main и develop и короткоживущими ветками feature, release и hotfix. *≈ Всего сделано свыше восьмидесяти коммитов по конвенции Conventional Commits с семантическим версионированием.* Трудоёмкость оценена методом PERT по пяти этапам, а календарный план отражён диаграммой Ганта.

Слайд 14 — Реализация: ключевые решения

≈ Остановлюсь на ключевых решениях реализации. Первое — загрузка мимо сервера: клиент пишет части файла напрямую в MinIO по подписанным ссылкам, а сервер лишь координирует сессию. Второе — наследование прав: доступ к папке распространяется на всё её поддерево, и эффективное право вычисляется одним рекурсивным запросом. Третье — фоновый worker с очередью задач в самой базе данных, конкурентным захватом и повторами. *≈ Два из этих решений покажу на примерах кода.*

Слайд 15 — Пример запроса: наследование прав

Наследование прав реализовано рекурсивным обобщённым табличным выражением. *≈ Слева — SQL-запрос: базовая часть выбирает сам узел, а рекурсивная поднимается по ссылке на родителя, пропуская удалённые узлы; затем отбираются активные права пользователя на любом из предков.* Справа — тот же запрос на SQLAlchemy. Такой подход устраняет антипаттерн «N+1 запрос».

Слайд 16 — Пример запроса: захват задач очереди

Фоновые задачи берутся из очереди запросом `SELECT FOR UPDATE SKIP LOCKED`. Он блокирует выбранные строки и пропускает уже занятые другими процессами, поэтому несколько воркеров работают одновременно без отдельного брокера и координатора. *⌘ Справа показана реализация на SQLAlchemy.*

Слайд 17 — Пользовательский интерфейс

⌘ Клиентская часть — адаптивное одностраничное приложение с тёмной и светлой темами. На слайде показаны файловый браузер с превью и панель администратора.

Слайд 18 — Тестирование и покрытие

Качество подтверждено автоматическими тестами: всего выполнено 7679 тестов — 6745 серверных и 934 клиентских, все пройдены. *⌘ Из серверных 6559 модульных и 186 интеграционных.* Покрытие кода составляет 98,6 процента для серверной части и 90,2 для клиентской. *⌘ Нагрузочное тестирование показало задержку p99 не выше 177 миллисекунд при требовании в две секунды; дополнительно проверены механизмы безопасности.*

Слайд 19 — Технико-экономическое обоснование

Методом PERT рассчитана полная себестоимость разработки — 802 тысячи рублей. *⌘ Поскольку продукт тиражируемый и распространяется с открытым исходным кодом, экономический эффект оценён как совокупная экономия владельцев развёртываний.* Годовая экономия одного развёртывания — 38 400 рублей, а дисконтированный срок окупаемости — 1,6 года. *⌘ Чистый дисконтированный доход за пять лет — около 5,8 миллиона рублей.*

Слайд 20 — Безопасность и охрана труда

В разделе безопасности жизнедеятельности рассмотрены организация рабочего места по ГОСТ, параметры микроклимата и освещения, а также вопросы электро- и пожарной безопасности. *⌘ Дополнительно определён режим труда и отдыха при работе с компьютером.*

Слайд 21 — Заключение

В заключение отмечу, что все поставленные задачи решены, а цель работы достигнута. Создан готовый к развёртыванию продукт, который благодаря передаче файлового трафика в обход сервера способен обслуживать файлы, размер которых многократно превышает объём его оперативной памяти. *⌘ Полученные архитектурные решения — подписанная многочастичная загрузка, очередь задач в СУБД и наследуемые права — применимы и за пределами данной задачи.*

Слайд 22 — Завершение

Доклад окончен. Благодарю за внимание и готов ответить на ваши вопросы.